COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	August 28, 2026	**DATE SUBMITTED:**	August 28, 2026
NAME:	DE VERA, Sophia D.	**INSTRUCTOR:**	Engr. QUEJADO, Jimlord

ACTIVITY NO. 6.1 SEARCHING ALGORITHM

1. PROCEDURE OUTPUT

Table 6-2a. Linear Search for Arrays

SCREENSHOT

main.cpp searchingAlgo.h node.h

main.cpp

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "searchingAlgo.h"

const int max_size = 50;

int main(){

    // generate random value
    std::cout << "      The generated fifty (50) numbers are:       " << std::endl;

    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

    int choice;
    std::cout << std::endl;
    std::cout << "Indicate the number to search: " << std::endl;
    std::cin >> choice;

    linearSearchArr(max_size, choice, dataset);
```

```
      The generated fifty (50) numbers are:
2057 10025 12192 16229 17541 18443 13371 15640 25894 25659 946 2009 27055 30591 31110 24760 21565 27537 18522 8285 2346 2163 30968 18938 26162 16131 23189 19070 21
716 463 18518 14348 23547 8234 26643 9643 29918 22400 18010 24383 10021 3814 29623 8632 12365 28917 1842 9097 14925 22489

Indicate the number to search:
2

The search is successful
```

OBSERVATION ANALYSIS

- **In this program, the code searches for the element in the list with the i (as index). From the randomated list, the program will look for the user input element in a linear motion. If the value matches, the function displays that the data was found and stops.**
- **In my opinion, this type of searching is best to use on a short list because it may take longer when the list contains many elements since each element needs to be checked one by one.**

Table 6-2b. Linear Search for Linked List

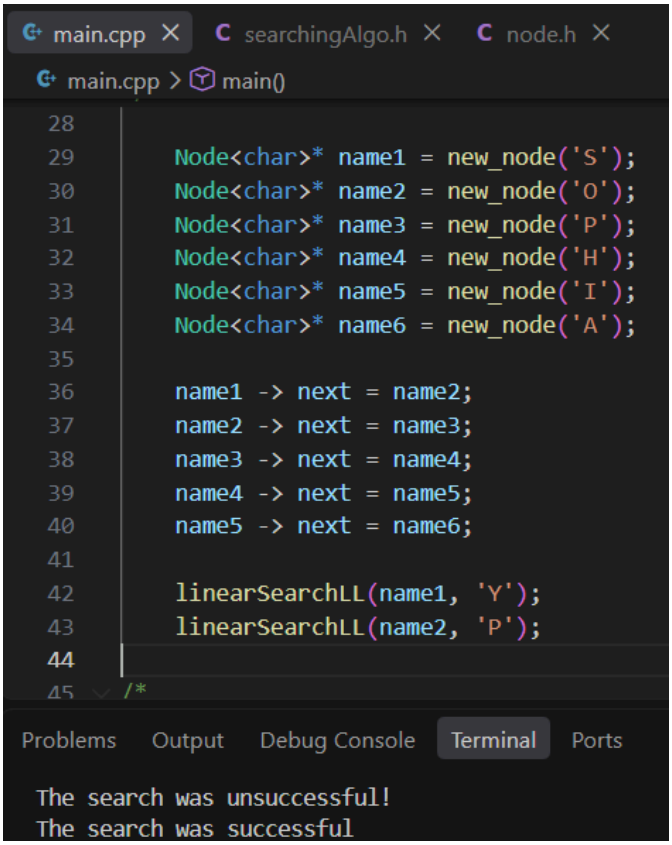
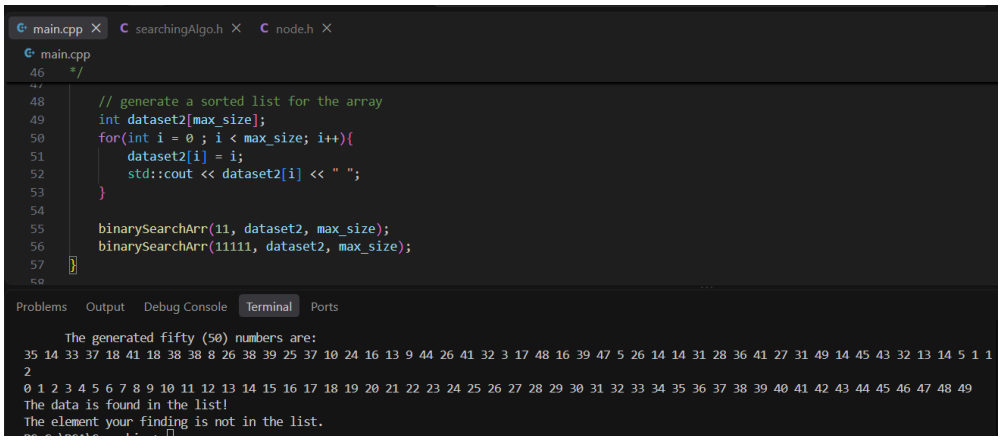
SCREENSHOT	
OBSERVATION ANALYSIS	- This is similar to the array searching based. But the difference is that linked lists move from one node to the next. It compares node->data with dataFind, and if they are equal,

Table 6-3a. Binary Search for Arrays

SCREENSHOT	

OBSERVATION ANALYSIS	- **Binary searching is a more efficient method than linear searching. From the word itself, binary divides the array into two parts using the low and up values. The program first searches the upper half. If it is smaller, it searches the lower half. This process continues until the element is found or there are no more elements to search.**

Table 6-3b. Binary Search for Linked List

SCREENSHOT	main.cpp × searchingAlgo.h × node.h × main.cpp > main() 8 int main(){ 59 // need to put in alphabetical order 60 Node<char>* name1 = new_node('A'); 61 Node<char>* name2 = new_node('D'); 62 Node<char>* name3 = new_node('E'); 63 Node<char>* name4 = new_node('H'); 64 Node<char>* name5 = new_node('I'); 65 Node<char>* name6 = new_node('O'); 66 Node<char>* name7 = new_node('P'); 67 Node<char>* name8 = new_node('R'); 68 Node<char>* name9 = new_node('S'); 69 Node<char>* name10 = new_node('V'); 70 Node<char>* name11 = new_node('_'); 71 72 name1 -> next = name2; 73 name2 -> next = name3; 74 name3 -> next = name4; 75 name4 -> next = name5; 76 name5 -> next = name6; 77 name6 -> next = name7; 78 name7 -> next = name8; 79 name8 -> next = name9; 80 name9 -> next = name10; 81 name10 -> next = name11; 82 83 binarySearchLL(name1, 'P'); 84 binarySearchLL(name2, '_'); 85 binarySearchLL(name3, 'V'); 86 binarySearchLL(name4, 'Y'); 87 binarySearchLL(name5, 'Z'); 88 } Problems Output Debug Console Terminal Ports The element is found in the list. The element is found in the list. The element is found in the list. The element your finding is not in the list. The element your finding is not in the list.
OBSERVATION ANALYSIS	- **We used a fast and slow method in this type of binary searching. The purpose of these two is the find the element in a fast or slow motion, allowing the program to find the element faster.**

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1

SCREENSHOT

SupProb1.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "supSearchingAlgo.h"

int main(){

    Node<int>* num1 = new_node(15);
    Node<int>* num2 = new_node(18);
    Node<int>* num3 = new_node(2);
    Node<int>* num4 = new_node(19);
    Node<int>* num5 = new_node(0);
    Node<int>* num6 = new_node(8);
    Node<int>* num7 = new_node(14);
    Node<int>* num8 = new_node(19);
    Node<int>* num9 = new_node(14);

    num1 -> next = num2;
    num2 -> next = num3;
    num3 -> next = num4;
    num4 -> next = num5;
    num5 -> next = num6;
    num6 -> next = num7;
    num7 -> next = num8;
    num8 -> next = num9;

    linearSearchLL(num1, 18);

    return 0;
}
```

supSearchingAlgo.h

```cpp
#include "node.h"
#ifndef SUPSEARCHINGALGO_H
#define SUPSEARCHINGALGO_H

// will use sequential search ll
template <typename T>
void linearSearchLL(Node<T>* node, T dataFind){

    int comparisons = 0;
    while (node != nullptr) {
        comparisons++;
        if (node->data == dataFind) {
            std::cout << "The data is found in the list!" << std::endl;
            std::cout << "Number of comparisons: " << std::endl;
            std::cout << comparisons << std::endl;
            return;
        }
        node = node->next;
    }

    std::cout << "The data is not found in the list." << std::endl;
    std::cout << "Number of comparisons: " << std::endl;
    std::cout << comparisons << std::endl;
}

#endif
```

```
The data is found in the list!
Number of comparisons:
2
```

OBSERVATION ANALYSIS

- Just like in the procedure activity, using the linear searching link list to find the desired output is done by checking the element in the given array one by one. It first starts with the first element then checks the succeeding elements until at the

	last node of the array list.

PROBLEM 2

SCREENSHOT	
OBSERVATION ANALYSIS	- In this problem, we were tasked to find how many '18' appeared in the list. Using the sequential search, it checks each node and evaluates whether that node equates to 18. If it does, the counter increments. Since the array has two '18s', the output thus said that the 18 appeared 2 times in the list.

PROBLEM 3

SCREENSHOT

Code:

```cpp
// Problem #3
int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int maxSize = 10;

std::cout << "Sorted List: ";

for (int i = 0; i < maxSize; i++) {
    std::cout << data[i] << " ";
}

std::cout << std::endl;

ArrBinSearch(8, data, maxSize);

return 0;
}
```

```cpp
//Problem #3
template <typename T>
void ArrBinSearch(T dataFind, T data[], int max_Size) {

    int bottom = 0, upper = max_Size - 1, midpoint;

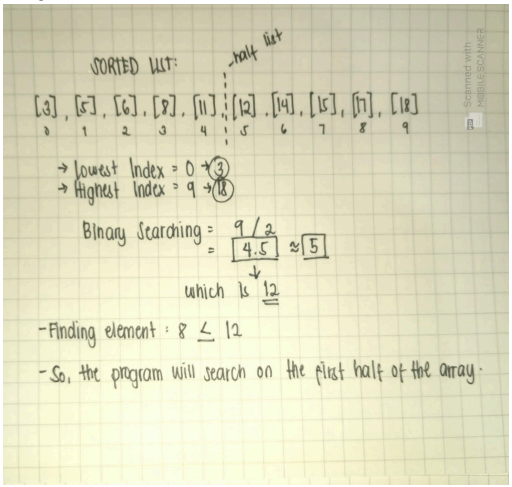
    while(bottom <= upper){
        midpoint = (int)((bottom + upper) / 2);

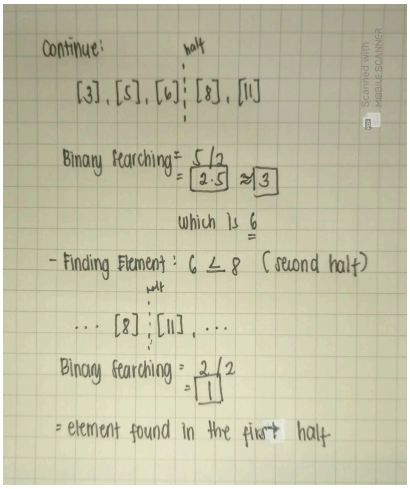
        if(dataFind == data[midpoint]){
            std::cout << "The data is in the list!" << std::endl;
            return;
        }
        else if(dataFind > data[midpoint]){
            bottom = midpoint + 1;
        }
        else{
            upper = midpoint - 1;
        }
    }

    std::cout << "No such element in the list." << std::endl;
}
```

```
Sorted List: 3 5 6 8 11 12 14 15 17 18
The data is in the list!
```

Diagram:

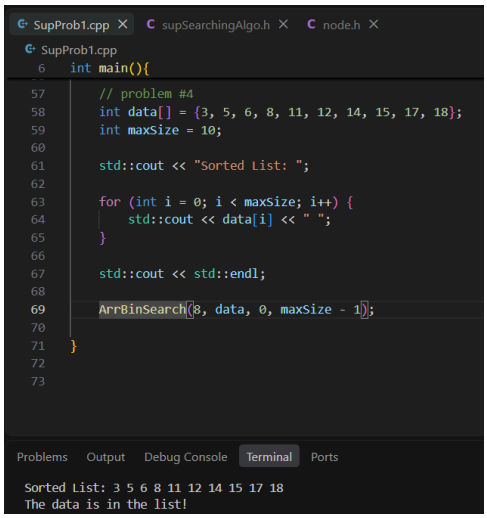
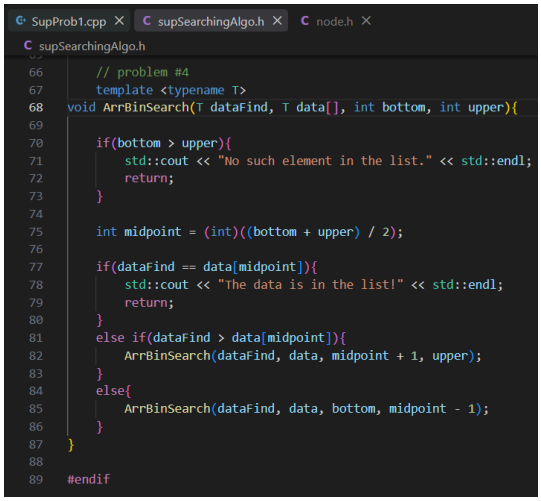




OBSERVATION ANALYSIS

- Using the binary searching algorithm, the data that we are looking for is the element '8'. Looking at the diagram I wrote, since its binary, the program divides the list into two parts (first and second half). Then it undergoes the equality comparison. If the midpoint is either greater or lower, the program adjusts depending on the answer in the value.

PROBLEM 4

SCREENSHOT	 
OBSERVATION ANALYSIS	In this problem, the binary search algorithm from Problem 3 was modified to become **recursive**. Instead of repeatedly using a loop, the function calls itself with a smaller search range depending on whether the target value is smaller or greater than the middle element.

CONCLUSION:

- **Learning the different searching algorithms, it is very important how to use each technique. Due to the reason that each technique method has their own efficient way to solve the searching methods. Take for example, in my opinion, searching in a very large sorted array, I would use a binary searching technique because it divides the list into halves, lessening the amount of time of running the code.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

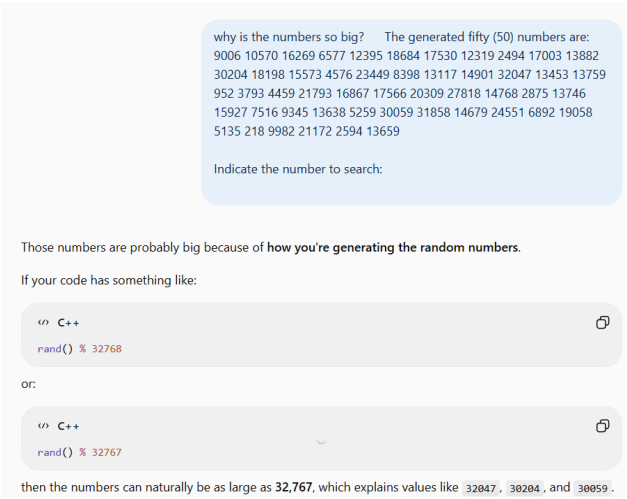
QUEZON CITY

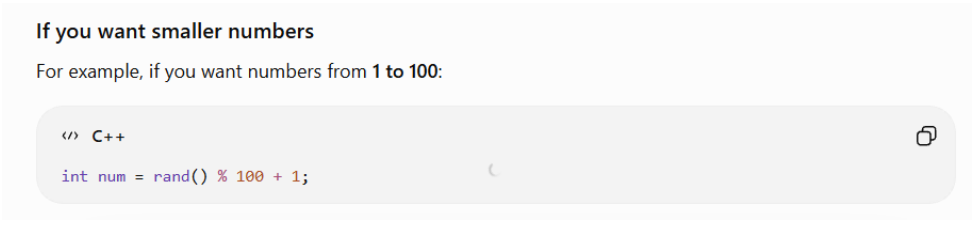
Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Asking for assistance.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.





Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

int main() {	template <typename T> void binarySearchArr(T dataFind, T data[], int

```
    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17,
18};

    binarySearchArr(8, data, 10);

    return 0;
}
```

```
maxSize) {

    int low = 0;
    int up = maxSize - 1;
    int middle;
    int comparisons = 0;

    while (low <= up) {

        middle = (low + up) / 2;
        comparisons++;

        if (dataFind == data[middle]) {

            std::cout << "The data is found!" <<
std::endl;
            std::cout << "Number of comparisons:
"
                    << comparisons << std::endl;
            return;
        }
        else if (dataFind > data[middle]) {

            low = middle + 1;
        }
        else {

            up = middle - 1;
        }
    }

    std::cout << "The data is not found!" <<
std::endl;
}
```

```
int main() {

    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17,
18};

    int result = binarySearchRecursive(data, 0,
9, 8);

    if (result) {
        std::cout << "The data is found!" <<
std::endl;
    }
```

```
template <typename T>
bool binarySearchRecursive(T data[], int low,
int up, T dataFind) {

    if (low > up) {
        return false;
    }

    int middle = (low + up) / 2;

    if (data[middle] == dataFind) {
        return true;
```

```
    else {
        std::cout << "The data is not found!" <<
std::endl;
    }

    return 0;
}
```

```
    }
    else if (dataFind < data[middle]) {
        return binarySearchRecursive(data, low,
middle - 1, dataFind);
    }
    else {
        return binarySearchRecursive(data,
middle + 1, up, dataFind);
    }
}
```